

Egison Core

Syntax and Typing Rules

1 構文

1.1 Capability, target type, scheme

$\kappa ::=$	$\bullet \mid \chi \mid \hat{s} \mid K \bar{\kappa} \mid (\kappa_1, \dots, \kappa_n)$	$\kappa : \text{Cap}$
$\tau ::=$	$\alpha \mid \hat{a} \mid \mathbf{1} \mid \text{Int} \mid \text{Bool} \mid K \bar{\tau} \mid (\tau_1, \dots, \tau_n) \mid \tau_1 \rightarrow \tau_2$ $\mid \text{Matcher } \kappa \tau \mid \text{MatcherSlot } \kappa \tau$	$\tau : \text{Type}$
$\sigma ::=$	$\forall(\bar{\chi} : \text{Cap}). \forall(\bar{\alpha} : \text{Type}). \tau$	expression scheme
$\delta ::=$	$(\kappa_1 \triangleright \tau_1), \dots, (\kappa_n \triangleright \tau_n) \Rightarrow (\kappa \triangleright \tau)$	pattern-function dual
$\varsigma ::=$	$\forall(\bar{\chi} : \text{Cap}). \forall(\bar{\alpha} : \text{Type}). \delta$	dual scheme
$\eta ::=$	$\kappa \triangleright \tau$	hole descriptor.

図1 二 sort の型構文. χ と α は flexible meta-variable である. $\hat{s}$ と $\hat{a}$ の rigid skolem は, 明示的な全称変数を型検査中だけ新鮮な固定定数として扱ったものであり, unification で別の capability / 型へ置換できない. $\text{mono } \tau$ は量化変数を一つも持たない単相 scheme を表し, 環境から取り出して instantiate しても新しい変数を作らず同じ τ を返す.

1.2 Source forms

$e ::=$	$x \mid \lambda x. e \mid e_1 e_2 \mid \text{let } x = e_1 \text{ in } e_2 \mid \text{fix } f x. e \mid n$ $\mid (e_1, \dots, e_n) \mid C \bar{e} \mid \text{op}(\bar{e}) \mid \text{something}$ $\mid \text{matcher } cls$ $\mid \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e$ $\mid \text{match } e_t \text{ as } e_m \text{ with } (p_i \rightarrow e_i)_i$
$p ::=$	$\$x \mid _ \mid \#e \mid \tilde{x} \mid c \bar{p} \mid p_1 \& p_2 \mid p_1 \mid p_2 \mid f \bar{p} \mid (p_1, \dots, p_n)$
$pp ::=$	$\$ \mid _ \mid \# \$x \mid c \bar{pp} \mid (pp_1, \dots, pp_n)$
$dp ::=$	$\$x \mid _ \mid C \bar{dp} \mid (dp_1, \dots, dp_n)$
$cl ::=$	$pp \text{ as } M \text{ with } [dp_j \rightarrow N_j]_{j=1}^r$
$cls ::=$	$[cl_i]_{i=1}^m$
$d ::=$	$\text{def pattern } f(x_1 : \tau_1, \dots, x_n : \tau_n) := p$
$\ell ::=$	$\text{none} \mid \chi \mid \hat{s}$
$ev ::=$	$\text{unseen} \mid \text{known } \ell \mid K \bar{ev} \mid (ev_1, \dots, ev_n).$

図2 c は pattern constructor, C は data constructor である. match は matchAll と head への surface elaboration であり, core primitive ではない.

1.3 Contexts and judgments

$\widehat{\Sigma} = (\widehat{\Sigma}_D, \widehat{\Sigma}_P, \widehat{\Sigma}_F, \widehat{\Sigma}_{prim}, Obs_{\widehat{\Sigma}})$	freeze 済み canonical signature
$\Gamma ::= \emptyset \mid \Gamma, x : \sigma$	expression scheme context
$\Phi ::= \emptyset \mid \Phi, x : \text{Pattern}(\kappa \triangleright \tau)$	pattern-parameter dual context
$\Delta ::= \emptyset \mid \Delta, x : \tau$	monomorphic pattern-variable context.
$\widehat{\Sigma}; \Gamma \vdash e : \tau$	expression typing
$\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p : \text{Pattern}(\kappa \triangleright \tau); \Delta'$	pattern dual typing
$\widehat{\Sigma} \vdash pp : \text{PPPattern } \tau \rightsquigarrow \bar{\eta}; \Delta_{pp}$	primitive-pattern hole/capture extraction
$\widehat{\Sigma}_D \vdash dp : \text{PDPattern } \tau \rightsquigarrow \Gamma_{dp}$	data-pattern binding
$\widehat{\Sigma}; \Gamma \vdash cl : \text{Clause } \tau \rightsquigarrow ev$	matcher-clause typing and evidence
$\widehat{\Sigma}; \Gamma \vdash d \text{ ok} \Rightarrow (f : \varsigma)$	pattern-function definition

図3 Primary judgments. Δ と Γ_{dp} を Γ に入れるときは各 binding を $\text{mono } \tau$ として扱う. $\uplus$ は domain-disjoint union, pattern の Δ は左から右へ thread する.

$\Gamma(x)$, $\Phi(x)$, $\widehat{\Sigma}_D(C)$ などは有限 map の lookup であり, key が存在しなければ失敗する. “fresh” は現在の signature, context, 型, capability のいずれにも出現しない同じ sort の変数を選ぶことを表す.

2 補助定義

この節の「部分関数」は成功値または失敗を返す. $f(x) = \text{some } y$ と $f(x) \Downarrow y$ はともに成功値が y であることを表し, $f(x) = \text{none}$ は失敗を表す. 判断規則の premise に現れる部分関数が失敗した場合, その規則は適用できない.

2.1 Instantiation, generalization, paired substitution

Capability substitution C と target substitution T は別 sort であり, $S = (C, T)$ の作用を次で定める.

$$S\kappa = C\kappa, \quad S\tau = T(C\tau), \quad S\Gamma = \{x : S\sigma \mid x : \sigma \in \Gamma\}.$$

Paired substitution $S = (C, T)$ は, すべての $\alpha \in \text{dom}(T)$ について $C(T\alpha) = T\alpha$ を満たすとき range-fixed である. 以下で S を構成する規則はこの条件を明示的に要求する.

$$\begin{aligned} \text{Inst}(\forall \bar{x}. \forall \bar{a}. \tau) &= (C_f, T_f)\tau, & C_f \bar{x}, T_f \bar{a} \text{ fresh}, \\ \text{Inst}(\text{mono } \tau) &= \tau, \\ \text{Gen}(\Gamma, \tau) &= \forall(\text{fcv}(\tau) \setminus \text{fcv}(\Gamma) : \text{Cap}). \forall(\text{ftv}(\tau) \setminus \text{ftv}(\Gamma) : \text{Type}). \tau, \\ \text{GenDual}(\Gamma, \delta) &= \forall(\text{fcv}(\delta) \setminus \text{fcv}(\Gamma) : \text{Cap}). \forall(\text{ftv}(\delta) \setminus \text{ftv}(\Gamma) : \text{Type}). \delta. \end{aligned}$$

Inst と InstDual は各 sort の全 binder を互いに異なる fresh flexible meta-variable へ同時に置換し, InstDual は dual scheme の両 binder 列を対象にする. Gen と GenDual の引数には prevailing paired substitution を適用し, 未解決の inference placeholder を含まない finalized type/capability を用いる. prevailing paired substitution とは, 同じ導出でそれ以前に得た全 constraint を解く累積的な一つの (C, T) である. Skolem は generalize しない. $\text{fcv}(X)$ と $\text{ftv}(X)$ はそれぞれ X に自由出現する flexible capability meta-variable と flexible target meta-variable の集合であり, scheme の binder に束縛された変数を含まない. $\text{dom}(X)$ は context または substitution X の定義域である.

Signature lookup は毎回 fresh instance を返すものとする.

$$\begin{aligned} \text{Inst}(\widehat{\Sigma}_D(C)) &= \tau_1 \times \cdots \times \tau_n \rightarrow \tau, \\ \text{Inst}(\widehat{\Sigma}_P(c)) &= (\tau_1, \dots, \tau_n) \Rightarrow_P \tau, \\ \text{InstDual}(\widehat{\Sigma}_F(f)) &= (\eta_1, \dots, \eta_n) \Rightarrow \eta, \\ \text{Inst}(\widehat{\Sigma}_{prim}(op)) &= \tau_1 \times \cdots \times \tau_n \rightarrow \tau. \end{aligned}$$

2.2 Equality, capability matching, signature operations

$$\begin{aligned} \text{mguTy}(\tau, \tau') &= T && \text{target-sort MGU,} \\ \text{mguCap}(\kappa, \kappa') &= C && \text{capability-sort MGU,} \\ \text{matchCap}(\kappa_m, \kappa_p) &= C && \text{dom}(C) \subseteq \text{fcv}(\kappa_p), \quad C\kappa_m = \kappa_m, \quad C\kappa_p = \kappa_p. \end{aligned}$$

matchCap は producer κ_m を rigid に読み, consumer demand κ_p の変数だけを解く most-general one-way match であり, 条件を満たす substitution がなければ失敗する. mguTy と mguCap はそれぞれの sort の決定的な部分関数であり, 成功時は most-general unifier を返す. constructor / arity の不一致, occurs check の失敗, または rigid skolem を解く必要がある場合は失敗する.

Frozen pattern signature が提供する二つの部分関数を次で表す.

$$\pi_c = \text{Inst}(\widehat{\Sigma}_P(c)), \quad \text{cap}_{\widehat{\Sigma}}(\pi_c; \bar{\kappa}) \Downarrow \kappa, \quad \text{ty}_{\widehat{\Sigma}}(\pi_c; \bar{\tau}) \Downarrow \tau'.$$

π_c はこの二つの部分関数へ明示的に渡す同一の fresh pattern-constructor signature instance である. cap はその field に $\bar{\kappa}$ を対応付け, result argument slot へ capability を投影して constructor pattern 全体の capability を返す. ty は $\bar{\tau}$ を field target と照合し, 同じ signature instance の result target を返す. いずれも arity, canonical former, または rigid constraint が一致しなければ失敗する. cap は $\text{Obs}_{\widehat{\Sigma}}$ が公開する signature path だけをたどり, target substitution や expected target から capability を生成しない.

2.3 Matcher-clause metafunctions

$$\begin{aligned} \text{prod}_0() &= \mathbf{1}, \\ \text{prod}_1(\tau_1) &= \tau_1, \\ \text{prod}_k(\tau_1, \dots, \tau_k) &= (\tau_1, \dots, \tau_k) \quad (k \geq 2), \\ \text{decomposeME}(\cdot, 0) &= [], \\ \text{decomposeME}(M, 1) &= [M], \\ \text{decomposeME}((M_1, \dots, M_k), k) &= [M_1, \dots, M_k] \quad (k \geq 2). \end{aligned}$$

上記以外の decomposeME は失敗する. $()$ は unit value, その型は $\mathbf{1}$ と同一視する. したがって $k = 0$ の matcher expression は $()$ だけである.

Complete capability を partial evidence へ埋め込む canonical map は次である.

$$\begin{aligned} [\bullet]_{ev} &= \text{known none}, & [\chi]_{ev} &= \text{known } \chi, \\ [\hat{s}]_{ev} &= \text{known } \hat{s}, & [K\bar{\kappa}]_{ev} &= K[\bar{\kappa}]_{ev}, \\ [(\kappa_1, \dots, \kappa_n)]_{ev} &= ([\kappa_1]_{ev}, \dots, [\kappa_n]_{ev}). \end{aligned}$$

$\text{clauseEvidence}_{\widehat{\Sigma}}(pp; [\kappa_1, \dots, \kappa_k])$ は, pp の hole を左から右へ並べた列と, それぞれの hole に実際に渡される next matcher の slot capability 列から, 一 clause 分の partial evidence を返す決定的な部分関数である. bare-hole catch-all \$, wildcard $_$, value pattern $\#\$x$ は root evidence を生成せず unseen を返す一方, constructor または tuple の内側にある \$ は対応する κ_i の canonical evidence を供給する. Tuple は component evidence をそのまま product node にし, constructor-headed pp は fresh/frozen signature の field type と result argument slot に沿って child evidence を投影する. このとき surface constructor 名ではなく signature の result former を evidence の root とする. hole 数, arity, canonical former, または同じ result slot へ届く evidence が exact に一致しなければ none を返す. target 型, expected 型, annotation は evidence の seed にしない. unseen は assignment を作らず, opaque / function / unobservable path は barrier として内部を観測しない. 必要な observable path に既知の異なる head が現れた場合は, unseen へ弱めず失敗する.

ClausesTy は actual clause list を順に検査し, 一 clause につき一つの evidence を同じ順序で返す output-indexed judgment である.

$$\text{ClausesTy}(\widehat{\Sigma}; \Gamma, [cl_i]_{i=1}^m, \tau) \Downarrow [ev_i]_{i=1}^m \iff \forall i. \widehat{\Sigma}; \Gamma \vdash cl_i : \text{Clause } \tau \rightsquigarrow ev_i.$$

いずれかの clause の PP extraction, next-matcher decomposition / slot typing, または一 clause 分の evidence projection が失敗すれば, この判断は成立しない.

2.4 Shape and coverage

Evidence の exact merge と shape inference は次で定める.

$$\begin{aligned} \text{merge}(\text{unseen}, ev) &= ev & \text{merge}(ev, \text{unseen}) &= ev, \\ \text{merge}(\text{known } \ell, \text{known } \ell) &= \text{known } \ell, & \text{merge}(K\bar{e}, K\bar{f}) &= K \text{ merge}(\bar{e}, \bar{f}), \\ \text{merge}((\bar{e}), (\bar{f})) &= (\text{merge}(\bar{e}, \bar{f})). \end{aligned}$$

Constructor name, node kind, arity, leaf identity が一致しない merge は失敗する. 上の式では成功時の some を省略している. 列上の merge は同じ長さの列を pointwise に merge し, 長さが異なるか一成分でも失敗すれば失敗する. mergeAll は unseen を初期値として evidence list を merge で畳み込む決定的な部分関数であり, 途中の merge が一度でも失敗すれば none を返す.

$$\text{inferShape}(Obs, \bar{ev}) = \begin{cases} \text{some } \bullet & \text{mergeAll}(\bar{ev}) = \text{some unseen}, \\ \text{finalize}_{Obs}(ev) & \text{mergeAll}(\bar{ev}) = \text{some } ev, ev \neq \text{unseen}, \\ \text{none} & \text{mergeAll}(\bar{ev}) = \text{none}. \end{cases}$$

finalize_{Obs} は partial evidence を complete capability へ変換する決定的な部分関数である. observable な unseen が残れば none を返し, unobservable な constructor parameter は $\bullet$ に canonicalize する. known none, known χ , known $\hat{s}$ はそれぞれ $\bullet$, χ , $\hat{s}$ に戻す. Constructor node は root former と同じ長さの frozen observability mask を用い, product component はすべて observable として再帰的に finalize する. 未知の mask や arity 不一致は失敗する.

$$\text{ShapeCap}_{\hat{\Sigma}}(\bar{ev}, \kappa) \iff \text{inferShape}(Obs_{\hat{\Sigma}}, \bar{ev}) = \text{some } \kappa.$$

T-MATCHER は ClausesTy が actual clause list から返した同じ $\bar{ev}$ を ShapeCap へ渡すため, 別の evidence list を選び直すことはできない. $Obs_{\hat{\Sigma}}(K)$ は canonical former K の parameter 数と同じ長さの Boolean mask を返し, 各 parameter が pattern-constructor signature から capability として観測可能かを示す. この mask は freeze 前に全 signature から計算した least fixpoint であり, 個々の matcher clause や target annotation には依存しない.

$$\begin{aligned} \text{CoverageOK}_{\hat{\Sigma}}(cls, \bullet) &\iff \text{true}, \\ \text{CoverageOK}_{\hat{\Sigma}}(cls, K\bar{\kappa}) &\iff \forall c \in \text{ctors}_{\hat{\Sigma}_P}(K). \exists cl \in cls. cl.pp = c \$ \dots \$, \\ \text{CoverageOK}_{\hat{\Sigma}}(cls, (\kappa_1, \dots, \kappa_n)) &\iff \exists cl \in cls. cl.pp = (\$, \dots, \$). \end{aligned}$$

CoverageOK のその他の root form は成立しない. Refinement は general clause に数えず, coverage は child capability へ再帰しない. $\text{ctors}_{\hat{\Sigma}_P}(K)$ は, fresh instantiate した result target の canonical root former が K である pattern constructor の有限集合であり, $cl.pp$ は clause cl の primitive-pattern-pattern を取り出す selector である. Formal core では CoverageOK を matcher literal の必須条件とする. Egison 本体は同じ検査の失敗を warning として扱ってよいが, その診断方針は core の外側に置く.

2.5 Matcher well-formedness predicates

$$\begin{aligned} \text{CatchAllLast}(cls) &\iff \exists \text{prefix}, M, x, N. cls = \text{prefix} ++ [\$ \text{ as } M \text{ with } [\$ x \rightarrow N]] \\ &\quad \wedge \forall cl \in \text{prefix}. cl.pp \neq \$, \\ \text{ArmExhaustive}_{\hat{\Sigma}_D}(cls, \tau) &\iff \forall cl \in cls. \text{Exhaustive}_{\hat{\Sigma}_D}(\text{arms}(cl), \tau), \\ \text{PPBindNodup}(cls) &\iff \forall cl \in cls. \text{NoDup}(\text{bindPP}(cl.pp)), \\ \text{ArmBindNodup}(cls) &\iff \forall cl \in cls. \forall dp \in \text{arms}(cl). (\text{NoDup}(\text{bindDP}(dp)) \\ &\quad \wedge \text{bindDP}(dp) \cap \text{bindPP}(cl.pp) = \emptyset). \end{aligned}$$

$\text{arms}(cl)$ は clause の $[dp_j \rightarrow N_j]_j$ から data pattern 列 $[dp_j]_j$ を順序を保って取り出す。 $\text{bindPP}(pp)$ と $\text{bindDP}(dp)$ はそれぞれ pp の $\#x$ と dp の $\$x$ が導入する変数名を左から右へ集め、 NoDup はその列に重複がないことを表す。 $\text{Exhaustive}_{\widehat{\Sigma}_D}(\overline{dp}, \tau)$ は, freeze 済み data signature の constructor / tuple / wildcard 規則に従って $\overline{dp}$ が τ の全値を被覆するかを判定する決定的な coverage checker である。 未知の constructor, arity 不一致, または未被覆の場合は成立しない。

3 Expression typing

$$\begin{array}{c}
\frac{\Gamma(x) = \sigma \quad \text{Inst}(\sigma) = \tau}{\widehat{\Sigma}; \Gamma \vdash x : \tau} \quad [\text{T-VAR}] \qquad \frac{\widehat{\Sigma}; \Gamma, x : \text{mono } \tau_1 \vdash e : \tau_2}{\widehat{\Sigma}; \Gamma \vdash \lambda x. e : \tau_1 \rightarrow \tau_2} \quad [\text{T-LAM}] \qquad \frac{\widehat{\Sigma}; \Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \widehat{\Sigma}; \Gamma \vdash e_2 : \tau_1}{\widehat{\Sigma}; \Gamma \vdash e_1 e_2 : \tau_2} \quad [\text{T-APP}] \qquad \frac{}{\widehat{\Sigma}; \Gamma \vdash n : \text{Int}} \quad [\text{T-LIT}] \\
\\
\frac{\widehat{\Sigma}; \Gamma \vdash e_1 : \tau_1 \quad \sigma = \text{Gen}(\Gamma, \tau_1) \quad \widehat{\Sigma}; \Gamma, x : \sigma \vdash e_2 : \tau_2}{\widehat{\Sigma}; \Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2} \quad [\text{T-LET}] \qquad \frac{\widehat{\Sigma}; \Gamma, f : \text{mono } (\tau_1 \rightarrow \tau_2), x : \text{mono } \tau_1 \vdash e : \tau_2}{\widehat{\Sigma}; \Gamma \vdash \text{fix } f x. e : \tau_1 \rightarrow \tau_2} \quad [\text{T-FIX}] \\
\\
\frac{\widehat{\Sigma}; \Gamma \vdash e_i : \tau_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma \vdash (e_1, \dots, e_n) : (\tau_1, \dots, \tau_n)} \quad [\text{T-TUPLE}] \\
\\
\frac{\text{Inst}(\widehat{\Sigma}_D(C)) = \tau_1 \times \dots \times \tau_n \rightarrow \tau \quad \widehat{\Sigma}; \Gamma \vdash e_i : \tau_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma \vdash C e_1 \dots e_n : \tau} \quad [\text{T-CON}] \\
\\
\frac{\text{Inst}(\widehat{\Sigma}_{\text{prim}}(op)) = \tau_1 \times \dots \times \tau_n \rightarrow \tau \quad \widehat{\Sigma}; \Gamma \vdash e_i : \tau_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma \vdash op(e_1, \dots, e_n) : \tau} \quad [\text{T-PRIM}] \qquad \frac{\alpha \text{ fresh in the target sort}}{\widehat{\Sigma}; \Gamma \vdash \text{something} : \text{Matcher} \bullet \alpha} \quad [\text{T-SOME}]
\end{array}$$

4 Pattern typing and pattern functions

$$\begin{array}{c}
\frac{x \notin \text{dom}(\Delta) \quad \chi, \alpha \text{ fresh}}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash \$x : \text{Pattern}(\chi \triangleright \alpha); \Delta, x : \alpha} \quad [\text{PAT-VAR}] \qquad \frac{\chi, \alpha \text{ fresh}}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash _ : \text{Pattern}(\chi \triangleright \alpha); \Delta} \quad [\text{PAT-WILD}] \\
\\
\frac{\widehat{\Sigma}; \Gamma, \text{mono } \Delta \vdash e : \tau \quad \chi \text{ fresh}}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash \#e : \text{Pattern}(\chi \triangleright \tau); \Delta} \quad [\text{PAT-VALUE}] \qquad \frac{\Phi(x) = \text{Pattern}(\kappa \triangleright \tau)}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash \widetilde{x} : \text{Pattern}(\kappa \triangleright \tau); \Delta} \quad [\text{PAT-EMBED}]
\end{array}$$

$$\frac{\widehat{\Sigma}; \Gamma; \Phi; \Delta_{i-1} \vdash p_i : \text{Pattern}(\kappa_i \triangleright \tau_i); \Delta_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma; \Phi; \Delta_0 \vdash (p_1, \dots, p_n) : \text{Pattern}((\kappa_1, \dots, \kappa_n) \triangleright (\tau_1, \dots, \tau_n)); \Delta_n} \quad [\text{PAT-TUPLE}]$$

$$\frac{\pi_c = \text{Inst}(\widehat{\Sigma}_P(c)) = (\rho_1, \dots, \rho_n) \Rightarrow_P \rho \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta_{i-1} \vdash p_i : \text{Pattern}(\kappa_i \triangleright \tau_i); \Delta_i \quad (1 \leq i \leq n) \quad \text{cap}_{\widehat{\Sigma}}(\pi_c; \bar{\kappa}) \Downarrow \kappa \quad \text{ty}_{\widehat{\Sigma}}(\pi_c; \bar{\tau}) \Downarrow \tau}{\widehat{\Sigma}; \Gamma; \Phi; \Delta_0 \vdash c p_1 \dots p_n : \text{Pattern}(\kappa \triangleright \tau); \Delta_n} \quad [\text{PAT-CON}]$$

$$\frac{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 : \text{Pattern}(\kappa \triangleright \tau); \Delta_1 \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta_1 \vdash p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta_2}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 \& p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta_2} \quad [\text{PAT-AND}]$$

$$\frac{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 : \text{Pattern}(\kappa \triangleright \tau); \Delta' \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta'}{\widehat{\Sigma}; \Gamma; \Phi; \Delta \vdash p_1 \mid p_2 : \text{Pattern}(\kappa \triangleright \tau); \Delta'} \quad [\text{PAT-OR}]$$

$$\frac{\text{InstDual}(\widehat{\Sigma}_F(f)) = (\kappa_1 \triangleright \tau_1), \dots, (\kappa_n \triangleright \tau_n) \Rightarrow (\kappa \triangleright \tau) \quad \widehat{\Sigma}; \Gamma; \Phi; \Delta_{i-1} \vdash p_i : \text{Pattern}(\kappa_i \triangleright \tau_i); \Delta_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma; \Phi; \Delta_0 \vdash f p_1 \dots p_n : \text{Pattern}(\kappa \triangleright \tau); \Delta_n} \quad [\text{PAT-APP}]$$

$$\frac{\begin{array}{l} f \notin \text{dom}(\widehat{\Sigma}_F) \quad \chi_1, \dots, \chi_n \text{ fresh} \\ \Phi' = \emptyset, x_1 : \text{Pattern}(\chi_1 \triangleright \tau_1), \dots, x_n : \text{Pattern}(\chi_n \triangleright \tau_n) \quad \widehat{\Sigma}; \Gamma; \Phi'; \emptyset \vdash p_{\text{body}} : \text{Pattern}(\kappa \triangleright \tau); \Delta \\ \widetilde{x_1}, \dots, \widetilde{x_n} \text{ occur exactly once, in declaration order, outside or-alternatives} \\ \varsigma_f = \text{GenDual}(\Gamma, (\chi_1 \triangleright \tau_1), \dots, (\chi_n \triangleright \tau_n) \Rightarrow (\kappa \triangleright \tau)) \end{array}}{\widehat{\Sigma}; \Gamma \vdash \text{def pattern } f(x_1 : \tau_1, \dots, x_n : \tau_n) := p_{\text{body}} \text{ ok} \Rightarrow (f : \varsigma_f)} \quad [\text{PATFUN-DEF}]$$

PATFUN-DEF は f を追加する前の $\widehat{\Sigma}_F$ で本体を検査する非再帰規則であり、同名の既存 pattern function を shadow しない。

5 Match and MatcherSlot

$$\frac{\widehat{\Sigma}; \Gamma \vdash e_t : \tau_t \quad \widehat{\Sigma}; \Gamma; \emptyset; \emptyset \vdash p : \text{Pattern}(\kappa_p \triangleright \tau_t); \Delta \quad \widehat{\Sigma}; \Gamma \vdash e_m : \text{MatcherSlot } \kappa_p \tau_t \quad \widehat{\Sigma}; \Gamma, \text{mono} \Delta \vdash e : \tau_r}{\widehat{\Sigma}; \Gamma \vdash \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e : \text{List } \tau_r} \quad [\text{T-MATCHALL}]$$

$$\frac{\widehat{\Sigma}; \Gamma \vdash e_t : \tau_t \quad \widehat{\Sigma}; \Gamma; \emptyset; \emptyset \vdash p_i : \text{Pattern}(\kappa_i \triangleright \tau_t); \Delta_i \quad (\forall i) \quad \widehat{\Sigma}; \Gamma \vdash e_m : \text{MatcherSlot } \kappa_i \tau_t \quad (\forall i) \quad \widehat{\Sigma}; \Gamma, \text{mono} \Delta_i \vdash e_i : \tau_r \quad (\forall i)}{\widehat{\Sigma}; \Gamma \vdash \text{match } e_t \text{ as } e_m \text{ with } (p_i \rightarrow e_i)_i : \tau_r} \quad [\text{T-MATCH (DERIVED)}]$$

$$\frac{\widehat{\Sigma}; \Gamma \vdash e : \text{Matcher } \kappa_m \tau_m \quad \text{matchCap}(\kappa_m, \kappa_p) = C \quad \text{mguTy}(C\tau_m, C\tau_t) = T \quad S = (C, T) \quad C(T\alpha) = T\alpha \quad (\alpha \in \text{dom}(T))}{\widehat{\Sigma}; ST \vdash e : \text{MatcherSlot } C\kappa_p S\tau_t} \quad [\text{COERCE-MATCHER-TO-SLOT}]$$

$$\frac{\widehat{\Sigma}; \Gamma \vdash e : \text{MatcherSlot } \kappa' \tau' \quad \text{mguCap}(\kappa', \kappa) = C \quad \text{mguTy}(C\tau', C\tau) = T \quad S = (C, T) \quad C(T\alpha) = T\alpha \quad (\alpha \in \text{dom}(T))}{\widehat{\Sigma}; ST \vdash e : \text{MatcherSlot } S\kappa S\tau} \quad [\text{CHECK-SLOT-TO-SLOT}]$$

$$\frac{\widehat{\Sigma}; \Gamma \vdash e_i : \text{Matcher } \kappa_i \tau_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}; \Gamma \vdash (e_1, \dots, e_n) : \text{Matcher } (\kappa_1, \dots, \kappa_n) (\tau_1, \dots, \tau_n)} \quad [\text{COERCE-TUPLE-MATCHER}] \quad \frac{\widehat{\Sigma}; \Gamma \vdash e : (\text{MatcherSlot } \kappa_1 \tau_1, \dots, \text{MatcherSlot } \kappa_n \tau_n)}{\widehat{\Sigma}; \Gamma \vdash e : \text{MatcherSlot } (\kappa_1, \dots, \kappa_n) (\tau_1, \dots, \tau_n)} \quad [\text{COERCE-SLOT-TUPLE}]$$

6 Matcher literals

6.1 Primitive pattern patterns

$$\frac{\chi \text{ fresh}}{\widehat{\Sigma} \vdash \$: \text{PPPattern } \tau \rightsquigarrow [\chi \triangleright \tau]; \emptyset} \quad [\text{PP-HOLE}] \quad \frac{}{\widehat{\Sigma} \vdash _ : \text{PPPattern } \tau \rightsquigarrow []; \emptyset} \quad [\text{PP-WILD}] \quad \frac{}{\widehat{\Sigma} \vdash \#\$x : \text{PPPattern } \tau \rightsquigarrow []; x : \tau} \quad [\text{PP-VAL}]$$

$$\frac{\text{Inst}(\widehat{\Sigma}_P(c)) = (\tau_1, \dots, \tau_n) \Rightarrow_P \tau \quad \widehat{\Sigma} \vdash pp_i : \text{PPPattern } \tau_i \rightsquigarrow \bar{\eta}_i; \Delta_i \quad (1 \leq i \leq n) \quad \bar{\eta} = \bar{\eta}_1 ++ \dots ++ \bar{\eta}_n}{\widehat{\Sigma} \vdash c pp_1 \dots pp_n : \text{PPPattern } \tau \rightsquigarrow \bar{\eta}; \Delta_1 \uplus \dots \uplus \Delta_n} \quad [\text{PP-CON}]$$

$$\frac{\widehat{\Sigma} \vdash pp_i : \text{PPPattern } \tau_i \rightsquigarrow \bar{\eta}_i; \Delta_i \quad (1 \leq i \leq n)}{\widehat{\Sigma} \vdash (pp_1, \dots, pp_n) : \text{PPPattern } (\tau_1, \dots, \tau_n) \rightsquigarrow \bar{\eta}_1 ++ \dots ++ \bar{\eta}_n; \Delta_1 \uplus \dots \uplus \Delta_n} \quad [\text{PP-TUPLE}]$$

6.2 Primitive data patterns

$$\frac{}{\widehat{\Sigma}_D \vdash \$x : \text{PDPattern } \tau \rightsquigarrow x : \tau} \quad [\text{PD-VAR}] \quad \frac{}{\widehat{\Sigma}_D \vdash _ : \text{PDPattern } \tau \rightsquigarrow \emptyset} \quad [\text{PD-WILD}]$$

$$\frac{\text{Inst}(\widehat{\Sigma}_D(C)) = \tau_1 \times \dots \times \tau_n \rightarrow \tau \quad \widehat{\Sigma}_D \vdash dp_i : \text{PDPattern } \tau_i \rightsquigarrow \Gamma_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}_D \vdash C dp_1 \dots dp_n : \text{PDPattern } \tau \rightsquigarrow \Gamma_1 \uplus \dots \uplus \Gamma_n} \quad [\text{PD-CON}]$$

$$\frac{\widehat{\Sigma}_D \vdash dp_i : \text{PDPattern } \tau_i \rightsquigarrow \Gamma_i \quad (1 \leq i \leq n)}{\widehat{\Sigma}_D \vdash (dp_1, \dots, dp_n) : \text{PDPattern } (\tau_1, \dots, \tau_n) \rightsquigarrow \Gamma_1 \uplus \dots \uplus \Gamma_n} \quad [\text{PD-TUPLE}]$$

6.3 Clauses and matcher construction

$$\begin{array}{c}
\widehat{\Sigma} \vdash pp : \text{PPPattern } \tau \rightsquigarrow [(\kappa_1 \triangleright \tau_1), \dots, (\kappa_k \triangleright \tau_k)]; \Delta_{pp} \quad \text{decomposeME}(M, k) = [M_1, \dots, M_k] \\
\widehat{\Sigma}; \Gamma \vdash M_l : \text{MatcherSlot } \kappa_l \tau_l \quad (1 \leq l \leq k) \quad \widehat{\Sigma}_D \vdash dp_j : \text{PDPattern } \tau \rightsquigarrow \Gamma_j \quad (1 \leq j \leq r) \\
\widehat{\Sigma}; \Gamma, \text{mono } \Delta_{pp}, \text{mono } \Gamma_j \vdash N_j : \text{List prod}_k(\tau_1, \dots, \tau_k) \quad (1 \leq j \leq r) \quad \text{clauseEvidence}_{\widehat{\Sigma}}(pp; [\kappa_1, \dots, \kappa_k]) = \text{some } ev \\
\hline
\widehat{\Sigma}; \Gamma \vdash (pp \text{ as } M \text{ with } [dp_j \rightarrow N_j]_{j=1}^r) : \text{Clause } \tau \rightsquigarrow ev
\end{array}$$

[CLAUSE-TY]

$$\begin{array}{c}
\text{ClausesTy}(\widehat{\Sigma}; \Gamma, cls, \tau) \Downarrow \overline{ev} \quad \text{ShapeCap}_{\widehat{\Sigma}}(\overline{ev}, \kappa) \quad \text{CatchAllLast}(cls) \\
\text{ArmExhaustive}_{\widehat{\Sigma}_D}(cls, \tau) \quad \text{PPBindNodup}(cls) \quad \text{ArmBindNodup}(cls) \quad \text{CoverageOK}_{\widehat{\Sigma}}(cls, \kappa) \\
\hline
\widehat{\Sigma}; \Gamma \vdash \text{matcher } cls : \text{Matcher } \kappa \tau
\end{array}$$

[T-MATCHER]